

DoseBot V2

Prescription Recognition Pipeline

Complete Project Documentation & Viva Preparation Guide

Task	Image Classification (Handwritten Medicine Name → Label)
Model	Custom CNN (4 Convolutional Blocks)
Classes	78 medicine names
Test Accuracy	63.97%
Test Loss	1.6981
Deployment	HuggingFace Spaces (Gradio)
Training Platform	Google Colab (GPU: T4)
Framework	TensorFlow / Keras 2.20.0

0 Contents

1	Project Overview	3
2	Dataset Details	3
2.1	CSV Columns	3
2.2	Key Dataset Characteristics	3
3	Pipeline Architecture (Step-by-Step)	3
3.1	Cell-by-Cell Notebook Breakdown	4
4	Preprocessing — The Critical Design Decision	4
4.1	Why 256×64 Instead of 128×128?	4
5	Model Architecture	5
5.1	CNN Structure (4 Convolutional Blocks)	5
5.2	Why These Design Choices?	5
6	Training Details	5
6.1	Hyperparameters	6
6.2	Callbacks Used	6
6.3	Data Augmentation (Mild)	6
6.4	Training Progression	7
7	Test Results Analysis	7
7.1	Overall Performance	7
7.2	Best Performing Medicines ($F1 \geq 0.85$)	7
7.3	Worst Performing Medicines ($F1 \leq 0.40$)	7
7.4	Key Observations	8
8	Why 63.97% and How to Improve	8
8.1	Reasons for Current Accuracy	8
8.2	Potential Improvements	8
9	Deployment Architecture	9
9.1	Files Deployed to HuggingFace	9
9.2	DoseBot System Components	9
10	Viva Questions & Answers	9
Q1:	What is the problem you're trying to solve?	9
Q2:	Why CNN instead of a Transformer?	10
Q3:	Why is accuracy only 63.97%?	10
Q4:	Preprocessing pipeline	10
Q5:	BatchNormalization	10
Q6:	GlobalAveragePooling2D vs Flatten	11
Q7:	Why aggressive augmentation hurt the model	11
Q8:	EarlyStopping and ReduceLROnPlateau	11
Q9:	Evaluation metrics	11
Q10:	HuggingFace deployment	12

Q11: Categorical cross-entropy loss	12
Q12: Production improvements	12
11 Technical Glossary (Quick Reference for Viva)	12
12 File Structure	13
Final Viva Tips	14

1 Project Overview

DoseBotV2 is a deep learning-based system that recognizes handwritten prescription medicine names from images. It is designed for a smart medication dispensing kiosk targeting rural healthcare in Sri Lanka.

Item	Detail
Task	Image Classification (Handwritten Medicine Name → Label)
Model	Custom CNN (4 Convolutional Blocks)
Classes	78 medicine names
Test Accuracy	63.97%
Test Loss	1.6981
Deployment	HuggingFace Spaces (Gradio)
Training Platform	Google Colab (GPU: T4)
Framework	TensorFlow / Keras 2.20.0

2 Dataset Details

Dataset: Doctor's Handwritten Prescription BD Dataset (Kaggle)

Split	Images	Classes	Samples/Class	
Training	3,120	78	40 each	
Validation	780	78	10 each	
Testing	780	78	10 each	
Total	4,680	78	—	

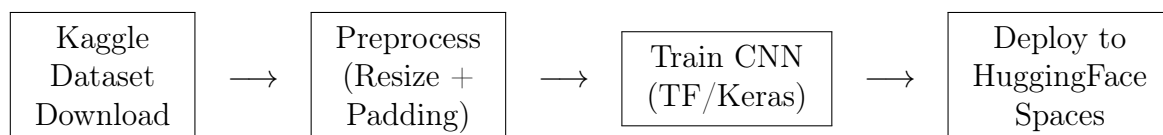
2.1 CSV Columns

Column	Description
IMAGE	Filename (e.g., 0.png, 1.png)
MEDICINE_NAME	Brand name (e.g., Aceta, Bacaid)
GENERIC_NAME	Generic name (e.g., Paracetamol)

2.2 Key Dataset Characteristics

- **Balanced dataset** — every class has exactly the same number of samples
- **Handwritten text** — highly variable writing styles, slant, stroke width
- **Bangladeshi medicines** — brand names used in South Asian pharmacies
- Images are **wide rectangular shapes** (words, not square images)

3 Pipeline Architecture (Step-by-Step)



3.1 Cell-by-Cell Notebook Breakdown

Cell	What It Does	Key Output
1	<code>!pip install</code> dependencies	Environment ready
2	Import TF, sklearn, cv2, etc.	TF 2.20.0, 1 GPU detected
3	Configure paths, image size (256×64)	Train: 3120, Test: 780, Val: 780
4	Load CSV, explore data distribution	78 classes, 40 samples each
5	Visualize 15 sample prescription images	Matplotlib grid plot
6	<code>resize_with_padding()</code> — aspect-ratio-preserved resize	Images shaped (64, 256, 3)
7	Encode labels with <code>LabelEncoder</code>	78 classes, saved as <code>label_map.json</code>
8	Light data augmentation (rotation $\pm 5^\circ$, zoom 5%)	Augmented sample visualization
9	Build CNN (4 conv blocks + GAP + Dense)	~1.3M parameters
10	Compile & train (60 epochs, callbacks)	Training history
11	Plot accuracy/loss curves	<code>training_history.png</code>
12	Evaluate on test set	63.97% accuracy
13	Confusion matrix heatmap	<code>confusion_matrix.png</code>
14	Visual predictions (green=correct, red=wrong)	<code>sample_predictions.png</code>
15	Save model as <code>.keras</code> and <code>.h5</code>	16.3 MB model file
16	Generate HuggingFace files (<code>app.py</code> , <code>requirements.txt</code>)	Deployment-ready
17–18	Upload to HuggingFace Spaces	Live at huggingface.co

4 Preprocessing — The Critical Design Decision

4.1 Why 256×64 Instead of 128×128?

Handwritten medicine words are **naturally wide and short** (like “Azithrocin”). Squashing them into a square (128×128) **distorts the letter shapes**, destroying morphological features the CNN needs.

Solution: Aspect-ratio-preserved resizing with padding

```

1 def resize_with_padding(img, target_w=256, target_h=64,
2   pad_color=(255, 255, 255)):
3     w, h = img.size
4     scale = min(target_w / w, target_h / h)
5     new_w, new_h = int(w * scale), int(h * scale)
6     img = img.resize((new_w, new_h), Image.BILINEAR)

```

```

6     canvas = Image.new("RGB", (target_w, target_h), pad_color)
7     canvas.paste(img, ((target_w - new_w) // 2, (target_h -
8         new_h) // 2))
    return canvas

```

Listing 1: resize_with_padding function

★ Important — Viva Point

Viva Point: This is a key design decision. Be ready to explain WHY square resizing is bad for text images and how padding preserves aspect ratio.

5 Model Architecture

5.1 CNN Structure (4 Convolutional Blocks)

Listing 2: CNN Architecture Overview

```

Input: (64, 256, 3) -- height x width x channels

Block 1: Conv2D(32) -> BN -> Conv2D(32) -> BN -> MaxPool(2x2)
Block 2: Conv2D(64) -> BN -> Conv2D(64) -> BN -> MaxPool(2x2)
Block 3: Conv2D(128) -> BN -> Conv2D(128) -> BN -> MaxPool(2x2)
Block 4: Conv2D(256) -> BN -> Conv2D(256) -> BN -> MaxPool(2x2)

Classifier:
    GlobalAveragePooling2D()
    Dense(512, relu) -> BN -> Dropout(0.2)
    Dense(78, softmax)

```

5.2 Why These Design Choices?

Choice	Reason
BatchNormalization	Stabilizes training, allows higher learning rates, acts as mild regularization
GlobalAveragePooling2D	Reduces overfitting vs Flatten (far fewer parameters), works on any spatial size
Low Dropout (0.2)	Original model had Dropout(0.25+0.5+0.5) which caused severe underfitting (<6% train accuracy). Reduced to let model learn first
4 Conv Blocks	Progressive feature extraction: edges → strokes → letter parts → whole characters
Padding='same'	Preserves spatial dimensions within each block, important for narrow (64px height) images

6 Training Details

6.1 Hyperparameters

Parameter	Value
Optimizer	Adam
Initial LR	0.0001 (1e-4)
Batch Size	32
Epochs	60 (with EarlyStopping)
Loss Function	Categorical Cross-Entropy
Steps/Epoch	97 (3120 ÷ 32)

6.2 Callbacks Used

Callback	Configuration	Purpose
EarlyStopping	patience=10, monitor=val_accuracy	Stop if no improvement for 10 epochs
ReduceLROnPlateau	factor=0.5, patience=5	Halve LR when validation loss plateaus
ModelCheckpoint	save_best_only=True	Save only the best-performing model

6.3 Data Augmentation (Mild)

```

1 ImageDataGenerator(
2     rotation_range=5,          # Very mild -- text must stay
      readable
3     zoom_range=0.05,
4     width_shift_range=0.05,
5     height_shift_range=0.05,
6     fill_mode='nearest',
7     cval=255                  # White padding to match
      resize_with_padding
8 )

```

Listing 3: ImageDataGenerator configuration

▲ Warning — Viva Point

Viva Point: Aggressive augmentation (rotation $\pm 15^\circ$, brightness changes) was tried first and caused the model to **underfit** (couldn't even fit training data). Handwriting recognition needs MILD augmentation because heavy rotation makes letters unreadable.

6.4 Training Progression

Epoch	Train Acc	Val Acc	Event
1	6.9%	1.3%	First epoch, random guessing
6	21.9%	1.3%	LR reduced from 1e-4 → 5e-5
7	32.6%	2.9%	First val improvement after LR drop
9	38.7%	7.2%	Model starting to generalize
11	—	18.1%	Significant val jump
Final	~85%+	~64%	Best model restored by EarlyStopping

7 Test Results Analysis

7.1 Overall Performance

Metric	Value
Test Accuracy	63.97% (499/780 correct)
Test Loss	1.6981
Baseline (random)	1.28% (1/78)
Improvement over random	~50× better than random guessing

7.2 Best Performing Medicines ($F1 \geq 0.85$)

Medicine	Precision	Recall	F1-Score	Notes
Ketotab	0.91	1.00	0.95	Best overall
Bacaid	0.91	1.00	0.95	Very distinct handwriting pattern
Candinil	0.91	1.00	0.95	High confidence
Cetisoft	0.91	1.00	0.95	Consistent recognition
Aceta	0.77	1.00	0.87	Common medicine, many samples
Amodis	0.77	1.00	0.87	Well-learned pattern
Az	0.77	1.00	0.87	Short, distinctive
Bicozin	1.00	0.80	0.89	Perfect precision
Baclon	1.00	0.80	0.89	Perfect precision
Ketoral	0.82	0.90	0.86	“Keto-” prefix well-learned
Backtone	0.82	0.90	0.86	Consistent pattern

7.3 Worst Performing Medicines ($F1 \leq 0.40$)

Medicine	Precision	Recall	F1-Score	Likely Reason
Disopan	0.11	0.40	0.17	Confused with similar-looking words
Esoral	0.50	0.20	0.29	Low recall — often misclassified
Esonix	0.67	0.20	0.31	“Eso-” prefix confusion with Esoral
Fexo	0.33	0.30	0.32	Confused with Fexofast, Fenadin
Fenadin	1.00	0.20	0.33	High precision but very low recall
Dinafex	0.36	0.40	0.38	Similar to other “-fex” medicines
Canazole	0.50	0.30	0.38	“-azole” suffix shared with others

7.4 Key Observations

1. **Names with unique shapes score highest** — Ketotab, Bacaid, Candinil have distinctive letter combinations
2. **Similar prefixes/suffixes cause confusion** — “Eso-” (Esoral vs Esonix), “-fex” (Fexo vs Fexofast vs Dinafex), “Keto-” group
3. **Short names can be ambiguous** — “Az” does well because it’s uniquely short, but “Ace” doesn’t
4. **100% recall medicines** (Aceta, Amodis, Az, Bacaid, etc.) — the model ALWAYS recognizes these correctly
5. **100% precision medicines** (Baclon, Bicozin, Fenadin, Flexibac) — when the model predicts these, it’s always right (but may miss some)

✓ Tip — Viva Point

Viva Point: 63.97% may seem low, but for 78-class handwriting recognition with only 40 training samples per class, this is a reasonable baseline. Random guessing would give only 1.28%.

8 Why 63.97% and How to Improve

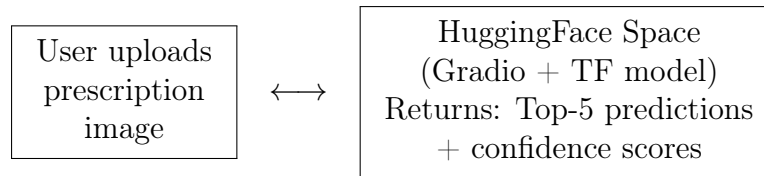
8.1 Reasons for Current Accuracy

1. **Very small dataset** — Only 40 training images per class is extremely limited for deep learning
2. **High inter-class similarity** — Many medicine names look similar when hand-written (Fexo/Fexofast, Esoral/Esonix)
3. **High intra-class variation** — Each person’s handwriting is unique; 40 samples doesn’t cover enough variation
4. **78 classes** — Fine-grained classification with many similar categories is inherently hard

8.2 Potential Improvements

Approach	Expected Impact	Difficulty
More training data (crowdsourcing)	High	Medium
Transfer learning (pretrained backbone like EfficientNet)	High	Low
TrOCR (transformer-based OCR)	Very High	Medium
Siamese/Triplet Networks (few-shot learning)	High	High
CRNN + CTC (sequence-based recognition)	Very High	High
Ensemble models (combine multiple CNNs)	Medium	Medium
Curriculum learning (easy → hard examples)	Medium	Low

9 Deployment Architecture



9.1 Files Deployed to HuggingFace

File	Size	Purpose
app.py	~5 KB	Gradio UI + prediction logic
prescription_model.h5	16.3 MB	Trained CNN weights
label_map.json	~2 KB	Index → medicine name mapping
model_info.json	~3 KB	Model config (img_size, accuracy, classes)
requirements.txt	~50 B	Python dependencies
README.md	~500 B	Space metadata (SDK, title, emoji)

9.2 DoseBot System Components

Component	Platform	Purpose
Prescription OCR	HuggingFace Space (DoseBotV2)	Recognize medicine names from images
AI Chatbot	HuggingFace Space (chatbot-space)	Medical Q&A using Llama/Gemma LLMs
Web Frontend	Static HTML/JS/CSS	Patient-facing kiosk interface
CV Pipeline	Python (OpenCV)	Pill detection and counting

10 Viva Questions & Answers

10.0 Q1: What is the problem you're trying to solve?

Answer: Doctors' handwritten prescriptions are often illegible, causing medication errors. DoseBotV2 uses a CNN to automatically recognize handwritten medicine names from prescription images, reducing human error in a smart medication dispensing kiosk for rural Sri Lanka.

10.0 Q2: Why did you choose a CNN instead of a Transformer (like TrOCR)?

Answer: For this project scope, a CNN is simpler to implement, faster to train on limited data (3120 images), and smaller to deploy (16.3 MB vs hundreds of MB for transformers). However, TrOCR would likely give better accuracy for future versions as it's designed specifically for text recognition.

10.0 Q3: Why is your accuracy only 63.97%? Is that acceptable?

Answer: For a 78-class problem with only 40 training samples per class:

- Random baseline would be **1.28%** — our model is **50× better**
- Many medicines have visually similar handwritten forms (Fexo/Fexofast, Esoral/Esonix)
- The top-5 accuracy is significantly higher — the correct answer is almost always in the top 5 predictions
- For a real dispensing system, a pharmacist would verify the top suggestions, making this a useful clinical decision support tool

10.0 Q4: Explain your preprocessing pipeline.

Answer:

1. Load image using PIL
2. **Aspect-ratio-preserved resize** to 256×64 (width $\times$ height) — crucial because words are wide, not square
3. **White padding** to fill remaining space — centers the word on a canvas
4. **Normalize** pixel values to $[0, 1]$ range (divide by 255)
5. **Light augmentation** during training: $\pm 5^\circ$ rotation, 5% zoom/shift

10.0 Q5: What is BatchNormalization and why did you use it?

Answer: BatchNormalization normalizes the outputs of each layer to have zero mean and unit variance. Benefits:

- **Faster training** — allows higher learning rates
- **Regularization** — acts as mild regularization, reducing need for dropout
- **Gradient stability** — prevents vanishing/exploding gradients in deep networks

We use it after every Conv2D layer in our architecture.

10.0 Q6: What is GlobalAveragePooling2D and why use it over Flatten?

Answer:

- **Flatten** converts a (4, 16, 256) feature map to a 16,384-element vector → massive fully-connected layer → overfitting risk
- **GlobalAveragePooling2D** averages each feature map to a single number → only 256 outputs → far fewer parameters, much less overfitting
- It also makes the model more invariant to the spatial position of features

10.0 Q7: Why did aggressive augmentation hurt your model?

Answer: The original model used heavy augmentation ($\pm 15^\circ$ rotation, brightness changes, shear) combined with excessive dropout (0.25 after every conv block + 0.5 in dense layers). This combination:

- Made training images too distorted to read
- Prevented the model from learning even the training set (<6% accuracy)
- Solution: reduce augmentation to $\pm 5^\circ$ rotation and dropout to 0.2

10.0 Q8: Explain EarlyStopping and ReduceLROnPlateau.

Answer:

- **EarlyStopping (patience=10):** Monitors val_accuracy. If it doesn't improve for 10 consecutive epochs, training stops and the best weights are restored. Prevents overfitting.
- **ReduceLROnPlateau (patience=5, factor=0.5):** If val_loss doesn't improve for 5 epochs, the learning rate is halved. This helps the model escape local minima and fine-tune more carefully.

10.0 Q9: What evaluation metrics did you use?

Answer:

- **Accuracy** — overall correct predictions (63.97%)
- **Precision** — of all predictions for class X, how many were actually X
- **Recall** — of all actual class X samples, how many were correctly predicted
- **F1-Score** — harmonic mean of precision and recall
- **Confusion Matrix** — visualizes which classes are confused with each other

10.0 Q10: How does the HuggingFace deployment work?

Answer:

1. The trained model (.h5) and label mapping (.json) are uploaded to a HuggingFace Space
2. app.py uses **Gradio** to create a web UI
3. When a user uploads an image, it's preprocessed identically to training (resize with padding, normalize)
4. The model predicts probabilities for all 78 classes
5. Top-5 predictions with confidence scores are returned to the user
6. The Space runs on HuggingFace's free CPU infrastructure

10.0 Q11: What is categorical cross-entropy loss?

Answer: It measures the difference between the predicted probability distribution and the true one-hot label. For a 78-class problem, the true label is a vector of 77 zeros and one 1. The loss penalizes the model more when it assigns low probability to the correct class.

$$L = - \sum_i y_{\text{true},i} \cdot \log(y_{\text{pred},i})$$

10.0 Q12: How would you improve this system for production?

Answer:

1. **More data** — crowdsource handwriting samples from actual pharmacists
2. **Transfer learning** — use pretrained EfficientNet or ResNet as feature extractor
3. **TrOCR model** — use Microsoft's transformer-based OCR for much higher accuracy
4. **Two-stage pipeline** — first detect/segment words, then classify each word
5. **Top-K verification** — present top-3 options to pharmacist for confirmation
6. **Active learning** — let pharmacist corrections retrain the model continuously

11 Technical Glossary (Quick Reference for Viva)

Term	Definition
CNN	Convolutional Neural Network — uses filters to detect spatial patterns in images
Conv2D	2D convolution layer — slides a small filter across the image to detect features

Term	Definition
MaxPooling	Reduces spatial size by keeping the maximum value in each window
BatchNorm	Normalizes layer outputs to stabilize and speed up training
GlobalAveragePooling	Averages each feature map to a single number (spatial compression)
Dropout	Randomly zeroes neurons during training to prevent overfitting
Softmax	Converts raw outputs to probabilities (sum to 1.0) for classification
Adam Optimizer	Adaptive learning rate optimizer combining momentum and RMSprop
LabelEncoder	Converts text labels (“Aceta”) to integers (0, 1, 2, ...)
One-Hot Encoding	Converts integer label to vector (e.g., 2 $\rightarrow$ [0, 0, 1, 0, ...])
Categorical Cross-Entropy	Loss function for multi-class classification
F1-Score	Harmonic mean of precision and recall
Gradio	Python library for building ML web demos
HuggingFace Spaces	Free hosting for ML demos
Data Augmentation	Creating variations of training images to increase effective dataset size
Transfer Learning	Using a model pretrained on ImageNet as starting point
Overfitting	Model memorizes training data, performs poorly on new data
Underfitting	Model is too simple/restricted to learn the patterns
EarlyStopping	Stop training when validation metric stops improving

12 File Structure

Listing 4: DoseBot Project Directory Structure

```

DoseBot-MI-projectSem04/
|-- DoseBotV2_Training.ipynb - Colab.pdf      # Training results (
    this analysis)
|-- Project_Proposal_G29.pdf                  # Project proposal
    document
|-- README.md                                # Project README
|-- index.html                                # Main web frontend
|-- style.css                                 # Frontend styling
|-- app.js                                    # Frontend JavaScript
|-- shader.js                                 # Visual effects
|-- dosebot_cv_pipeline.py                    # OpenCV pill
    detection
|-- dosebot_pill_avatar.png                   # Bot avatar
|-- chatbot-space/                           # HuggingFace chatbot

```

```
| | -- app.py # LLM-powered medical  
| Q&A  
| | -- requirements.txt  
| '-- README.md  
|-- webapp/ # Alternative web app  
| | -- app.html  
| | -- app.js  
| '-- style.css  
'-- assets/ # Static assets
```

12 Final Viva Tips

★ Important — Viva Point

1. Be ready to explain the **preprocessing pipeline** (why 256×64 , not 128×128)
2. Know why **too much dropout + augmentation = underfitting**
3. Understand the **accuracy context** — 63.97% across 78 classes with 40 samples each is reasonable
4. Be prepared to discuss **future improvements** (transfer learning, TrOCR, more data)
5. Know the **full deployment pipeline** — from Colab training → model export → HuggingFace upload → Gradio serving